## The process — one write, one read, one open

Integrity is enforced at the moment data is used, not audited afterwards. Nothing here is a background job that could be behind.

### A write

saving one drawer

1. **seal the content**
   zstd first, then encrypt — ciphertext has no redundancy left to compress, so the order matters.
2. **tag the row**
   `HMAC(mac_key, id || meta_json || sealed)`
   The tag covers the bytes as stored, ciphertext included.
3. **advance the chain**
   `head' = HMAC(mac_key, head || tag)`
   Each head commits to every write that came before it.
4. **ONE SQLite transaction**
   the row, the new chain head, and the chain entry commit together — or none of them do. There is no window where data exists without its audit entry.
5. **only then, the manifest anchor**
   Written to a temp file, atomically renamed, and the directory fsynced. It deliberately lags the database.

   Bulk ingest keeps this shape: one transaction and one anchor per batch, not per drawer.

### A read

every single one

1 · load the row exactly as stored

2 · recompute the tag over those bytes

3 · compare

mismatch ⇒ the row is refused and never becomes data

4 · only now decrypt, and return the words

Verification precedes use — it is not a separate verify pass.

### An open — reconciling the anchor

Compare the lagging manifest anchor with the committed head:

anchor BEHIND the database
a crash between commit and anchor — fast-forward, benign

anchor AHEAD of the database
the database went backwards — rollback, raise the alarm

This is the whole reason the anchor must never lead. If it could, an ordinary power loss would be indistinguishable from someone restoring an older copy of the file.

### Durability is pinned, not assumed — the ordering above only means something if the writes actually land

SQLite runs `WAL` + `synchronous=FULL` on both binaries, so a committed transaction has reached the disk before the call returns.

The manifest anchor is fsynced through an atomic rename plus a directory sync, so it is never half-written — a torn anchor would read as corruption on a vault that is in fact intact.

Key material is fsynced at creation. A key that is not durable is a vault nobody can open again, which is data loss wearing a security hat.
The invariant tying it together: the anchor may lag the database and must never lead it.

### What this detects, and what it does not

caught

An edited row — the tag no longer matches its bytes.
A deleted or reordered history — the chain will not replay.
A restored older database file — the anchor leads.
A blob moved between vaults or rows — the AAD fails.

not caught — and worth saying plainly

Anyone holding the master key can rewrite history and re-tag it so that it verifies. The chain proves the file has not been altered by someone without the key — it is not a defence against the key holder, and no local scheme could be. External anchoring is what would close that, and it is not built.